



PROBLEM SESSION 2

Process Creation & Pipes.

คณะผู้จัดทำ

นัธวัฒน์	ปริมสิริคุณาวุฒิ	67070501027
วิศิษฐ์	สุวรรณเนาวิ	67070501042
พลวิษฐ์	วัฒนเหมรัตน์	67070501067

เสนอ

ผศ. ราชวิช สโรชวิกสิต

รายงานนี้เป็นส่วนหนึ่งของรายวิชา

CPE333 ระบบปฏิบัติการ (Operating Systems)

ภาควิชาวิศวกรรมคอมพิวเตอร์ คณะวิศวกรรมศาสตร์

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี

ภาคเรียนที่ 1 ปีการศึกษา 2569

Problem 1: Process Creation with fork() and wait()

1. Write two programs as `fork()` example from process lecture slide:

1.1. The “hello world” one

1.2. The one that modified with `wait()` function

Run the program, observe and discuss the difference in the output between the two programs.

1.1 โปรแกรม fork() แบบ Hello World

Source code

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[]){
    printf("hello world (pid:%d)\n", (int)getpid());

    int rc = fork();

    if (rc < 0){
        fprintf(stderr, "fork failed\n");
        exit(1);
    }
    else if (rc == 0){
        printf("hello, I am child (pid:%d)\n", (int)getpid());
    }
    else{
        printf("hello, I am parent of %d (pid:%d)\n", rc, (int)getpid());
    }

    return 0;
}
```

หลักการทำงานของโปรแกรม (Program Operation Description)

โปรแกรมเริ่มต้นด้วย Process เดียวและแสดงข้อความ “hello world” พร้อม PID ก่อนเรียก `fork()` จากนั้น `fork()` จะสร้าง Child Process ขึ้นมาอีกหนึ่ง Process โดยใน Child ค่า return ของ `fork()` จะเป็น 0 ส่วนใน Parent ค่า return จะเป็น PID ของ Child ที่เพิ่งถูกสร้างขึ้น ทำให้โปรแกรมสามารถแยกเส้นทางการทำงานของ Parent และ Child ได้จากค่า rc

หลังจาก `fork()` แล้ว Parent และ Child เป็น Process ที่สามารถถูก Scheduler เลือกให้ทำงานได้อย่างอิสระ ดังนั้นลำดับการแสดงผลของบรรทัด Parent และ Child ไม่ได้ถูกกำหนดตายตัว แม้ในการทดลองครั้งนี้จะพบว่า Parent แสดงผลก่อน Child ทุกครั้งที่บันทึกไว้ก็ตาม

ผลการทดลอง (Output Results)

จากการรันโปรแกรมหลายครั้ง พบว่า “hello world” จะแสดงเพียงหนึ่งครั้งต่อการรัน เนื่องจากคำสั่งดังกล่าวอยู่ก่อน `fork()` ส่วนหลัง `fork()` จะมีทั้ง Parent และ Child แสดงผล โดย PID ของ Child ตรงกับค่าที่ Parent แสดงในข้อความ “parent of 13463”, “parent of 13466”, “parent of 13469”, “parent of 13472”, และ “parent of 13479”

```
wisitsuwannao@Ubuntu26LTS:~/CPE333/PS2/q1$ ./fork_basic
hello world (pid:13462)
hello, I am parent of 13463 (pid:13462)
hello, I am child (pid:13463)
wisitsuwannao@Ubuntu26LTS:~/CPE333/PS2/q1$ ./fork_basic
hello world (pid:13465)
hello, I am parent of 13466 (pid:13465)
hello, I am child (pid:13466)
wisitsuwannao@Ubuntu26LTS:~/CPE333/PS2/q1$ ./fork_basic
hello world (pid:13468)
hello, I am parent of 13469 (pid:13468)
hello, I am child (pid:13469)
wisitsuwannao@Ubuntu26LTS:~/CPE333/PS2/q1$ ./fork_basic
hello world (pid:13471)
hello, I am parent of 13472 (pid:13471)
hello, I am child (pid:13472)
wisitsuwannao@Ubuntu26LTS:~/CPE333/PS2/q1$ ./fork_basic
hello world (pid:13478)
hello, I am parent of 13479 (pid:13478)
hello, I am child (pid:13479)
```

รูปที่ 1 ผลการรัน `fork()` โดยไม่ใช้ `wait()` หลายครั้ง

Discussion

ผลที่ Parent แสดงก่อน Child ในรอบที่ทดลองเป็นเพียงพฤติกรรมที่สังเกตได้จาก Scheduler ในช่วงเวลานั้น ไม่ได้หมายความว่า Parent ต้องทำงานก่อน Child เสมอ เพราะระบบปฏิบัติการสามารถเลือก Process ใดก็ได้ผ่าน CPU ก่อนก็ได้หลังจาก `fork()`

1.2 โปรแกรม fork() ร่วมกับ wait()

Source code

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(int argc, char *argv[]){
    printf("hello world (pid:%d)\n", (int)getpid());

    int rc = fork();

    if (rc < 0){
        fprintf(stderr, "fork failed\n");
        exit(1);
    }
    else if (rc == 0){
        printf("hello, I am child (pid:%d)\n", (int)getpid());
    }
    else{
        wait(NULL);
        printf("hello, I am parent of %d (pid:%d)\n", rc, (int)getpid());
    }

    return 0;
}
```

หลักการทำงานของโปรแกรม (Program Operation Description)

โปรแกรมยังสร้าง Child ด้วย `fork()` เหมือนข้อ 1.1 แต่ในเส้นทางของ Parent มีการเรียก `wait(NULL)` ก่อนแสดงข้อความของ Parent ทำให้ Parent ต้องหยุดรอจนกว่า Child จะสิ้นสุดการทำงานก่อน แล้วจึงดำเนินคำสั่งหลัง `wait()` ต่อได้

ผลการทดลอง (Output Results)

จากการรันหลายครั้ง Child แสดงข้อความก่อน Parent ทุกครั้ง ซึ่งแตกต่างจากข้อ 1.1 ที่ลำดับหลัง `fork()` ไม่ได้รับการรับประกัน

```

wisitsuwannao@Ubuntu26LTS:~/CPE333/PS2/q1$ ./fork_wait
hello world (pid:14509)
hello, I am child (pid:14510)
hello, I am parent of 14510 (pid:14509)
wisitsuwannao@Ubuntu26LTS:~/CPE333/PS2/q1$ ./fork_wait
hello world (pid:14512)
hello, I am child (pid:14513)
hello, I am parent of 14513 (pid:14512)
wisitsuwannao@Ubuntu26LTS:~/CPE333/PS2/q1$ ./fork_wait
hello world (pid:14515)
hello, I am child (pid:14516)
hello, I am parent of 14516 (pid:14515)
wisitsuwannao@Ubuntu26LTS:~/CPE333/PS2/q1$ ./fork_wait
hello world (pid:14518)
hello, I am child (pid:14519)
hello, I am parent of 14519 (pid:14518)
wisitsuwannao@Ubuntu26LTS:~/CPE333/PS2/q1$ ./fork_wait
hello world (pid:14522)
hello, I am child (pid:14523)
hello, I am parent of 14523 (pid:14522)
wisitsuwannao@Ubuntu26LTS:~/CPE333/PS2/q1$ ./fork_wait
hello world (pid:14525)
hello, I am child (pid:14526)
hello, I am parent of 14526 (pid:14525)

```

รูปที่ 2 ผลการรัน `fork()` ร่วมกับ `wait()` ซึ่ง Child แสดงผลก่อน Parent

Discussion 1.1 และ 1.2

เมื่อไม่ใช้ `wait()` ลำดับการแสดงผลระหว่าง Parent และ Child ขึ้นกับการ Scheduling ของระบบปฏิบัติการ แต่เมื่อ Parent เรียก `wait()` Parent จะไม่สามารถผ่านจุดดังกล่าวไปยัง `printf()` ได้จนกว่า Child จะ terminate ดังนั้นลำดับ Child ก่อน Parent จึงถูกกำหนดโดย synchronization ที่เกิดจาก `wait()`

Problem 2: Process Termination, Zombie and Re-parenting

2. With help of `sleep()` function, you can make sure parents die before child do, and of course you can make sure parents die after child. Use `ps` command to examine the process status (or you can also use `ps -ef` to see parent of each process). Examine the process list when:

2.1. Child is dead while parent is still running.

2.2. Parents are dead while child is running.

Discuss the differences between 2 scenarios. Report the child's parent process ID in both scenarios. Explain the meaning, consequences, and causes of processes tagged as `<defunct>`.

2.1 Child สิ้นสุดก่อน ขณะที่ Parent ยังทำงานอยู่

Source code

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>

int main(void){
    pid_t pid = fork();

    if (pid < 0){
        perror("fork");
        exit(1);
    }
    else if (pid == 0){
        printf("Child: PID=%d, PPID=%d\n", (int)getpid(), (int)getppid());
        printf("Child: exiting now\n");
        fflush(stdout);
        exit(0);
    }
    else{
        printf("Parent: PID=%d, Child PID=%d\n", (int)getpid(), (int)pid);
        printf("Parent: sleeping for 30 seconds...\n");
        printf("Check the process status using ps in another terminal.\n");
        fflush(stdout);
        sleep(30);
        printf("Parent: exiting now without wait()\n");
    }

    return 0;
}
```

หลักการทำงานของโปรแกรม

Child แสดง PID และ PPID แล้วเรียก exit(0) ทันที ส่วน Parent ไม่ได้เรียก wait() และ sleep เป็นเวลา 30 วินาที จึงเกิดช่วงเวลา Child สิ้นสุดแล้วแต่ Parent ยังมีชีวิตอยู่ ทำให้สามารถตรวจสอบสถานะของ Child ผ่าน ps ได้

คำสั่งที่ใช้ตรวจสอบ

```
ps -o pid,ppid,state,stat,cmd -p <PARENT_PID>,<CHILD_PID>
```

ผลการทดลอง

Parent PID = 16487, Child PID = 16488 และ Child มี PPID = 16487 จาก ps พบ Child อยู่ใน State Z, STAT Z+ และมีข้อความ <defunct> ขณะที่ Parent ยังอยู่ใน State S เนื่องจากกำลัง sleep

```
wisitsuwannao@Ubuntu26LTS:~/CPE333/PS2/q2$ ./parent_dies_first
Parent: PID=16746, Child PID=16747
Parent: sleeping for 2 seconds before exiting...
Child: PID=16747, initial PPID=16746
Child: sleeping for 10 seconds...
Parent: exiting now
wisitsuwannao@Ubuntu26LTS:~/CPE333/PS2/q2$ Child: PID=16747, PPID after parent exits=4005
□
```



```
wisitsuwannao@Ubuntu26LTS: ~
[+]
wisitsuwannao@Ubuntu26LTS: ~/Desktop$ cd ..
wisitsuwannao@Ubuntu26LTS: ~$ ps -o pid,ppid,state,stat,cmd -p 15000,15001
  PID   PPID  S  STAT CMD
 15000     2  I   I    [kworker/0:0]
wisitsuwannao@Ubuntu26LTS: ~$ ps -o pid,ppid,state,stat,cmd -p 16359,16360
  PID   PPID  S  STAT CMD
wisitsuwannao@Ubuntu26LTS: ~$ ps -o pid,ppid,state,stat,cmd -p 16389,16390
  PID   PPID  S  STAT CMD
wisitsuwannao@Ubuntu26LTS: ~$ ps -o pid,ppid,state,stat,cmd -p 16487,16488
  PID   PPID  S  STAT CMD
 16487  12777  S  S+   ./child_dies_first
 16488  16487  Z  Z+   [child_dies_firs] <defunct>
wisitsuwannao@Ubuntu26LTS: ~$ ps -o pid,ppid,state,stat,cmd -p 16747
  PID   PPID  S  STAT CMD
```

รูปที่ 3 สถานะ Zombie (Z / <defunct>) เมื่อ Child สิ้นสุดก่อน Parent

Discussion

สถานะ Z หรือ Zombie หมายถึง Child สิ้นสุดการ execute แล้ว แต่ Parent ยังไม่ได้เรียก wait() หรือ waitpid() เพื่อเก็บ termination status ของ Child จึงยังมี Process Table Entry บางส่วนคงอยู่ เมื่อ Zombie เกิดเพียงเล็กน้อยจะไม่ใช้ CPU ในการประมวลผล แต่หากมีจำนวนมากจะกิน PID/Process Table resources และอาจส่งผลกระทบต่อความสามารถในการสร้าง Process ใหม่ได้ สาเหตุใน experiment นี้เกิดจาก Child จบก่อนและ Parent จงใจไม่เรียก wait()

2.2 Parent สิ้นสุดก่อน ขณะที่ Child ยังทำงานอยู่

Source code

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>

int main(void){
    pid_t pid = fork();

    if (pid < 0){
        perror("fork");
        exit(1);
    }
    else if (pid == 0){
        printf("Child: PID=%d, initial PPID=%d\n", (int)getpid(),
(int)getppid());
        fflush(stdout);

        printf("Child: sleeping for 10 seconds...\n");
        fflush(stdout);
        sleep(10);

        printf("Child: PID=%d, PPID after parent exits=%d\n", (int)getpid(),
(int)getppid());
        return 0;
    }
    else{
        printf("Parent: PID=%d, Child PID=%d\n", (int)getpid(), (int)pid);
        printf("Parent: sleeping for 2 seconds before exiting...\n");
        fflush(stdout);
        sleep(2);
        printf("Parent: exiting now\n");
        return 0;
    }
}
```

หลักการทำงานของโปรแกรม

Parent **sleep** เพียง 2 วินาทีแล้วสิ้นสุด ส่วน Child **sleep** 10 วินาที ทำให้ Parent เติมจบก่อน Child ระหว่างที่ Child ยังทำงานอยู่ ระบบจึงต้องทำ Re-parenting เพื่อให้ Child มี Parent Process ใหม่

ผลการทดลอง

ในการทดลอง Parent **PID** = 16799 และ Child **PID** = 16800 โดย Child มี initial **PPID** = 16799 เมื่อ Parent สิ้นสุดแล้วใช้ **ps** ตรวจสอบ Child อีกครั้งพบว่า Child ยังอยู่ใน State S แต่ **PPID** เปลี่ยนเป็น 4005 แสดงว่าเกิด Re-parenting จริงบน environment ที่ใช้ทดลอง

```
wisitsuwannao@Ubuntu26LTS:~/CPE333/PS2/q2$ ./parent_dies_first
Parent: PID=16799, Child PID=16800
Parent: sleeping for 2 seconds before exiting...
Child: PID=16800, initial PPID=16799
Child: sleeping for 10 seconds...
Parent: exiting now
wisitsuwannao@Ubuntu26LTS:~/CPE333/PS2/q2$
```

```
wisitsuwannao@Ubuntu26LTS: ~
wisitsuwannao@Ubuntu26LTS:~/Desktop$ cd ..
wisitsuwannao@Ubuntu26LTS:~$ ps -o pid,ppid,state,stat,cmd -p 15000,15001
  PID   PPID S  STAT CMD
 15000     2 I   I    [kworker/0:0]
wisitsuwannao@Ubuntu26LTS:~$ ps -o pid,ppid,state,stat,cmd -p 16359,16360
  PID   PPID S  STAT CMD
 16359   163 S  S+   ./child_dies_first
wisitsuwannao@Ubuntu26LTS:~$ ps -o pid,ppid,state,stat,cmd -p 16389,16390
  PID   PPID S  STAT CMD
 16389   163 S  S+   ./child_dies_first
wisitsuwannao@Ubuntu26LTS:~$ ps -o pid,ppid,state,stat,cmd -p 16487,16488
  PID   PPID S  STAT CMD
 16487   127 S  S+   ./child_dies_first
 16488   164 Z  Z+   [child_dies_firs] <defunct>
wisitsuwannao@Ubuntu26LTS:~$ ps -o pid,ppid,state,stat,cmd -p 16747
  PID   PPID S  STAT CMD
 16747   167 S  S+   ./child_dies_first
wisitsuwannao@Ubuntu26LTS:~$ ps -o pid,ppid,state,stat,cmd -p 16800
  PID   PPID S  STAT CMD
 16800   400 S  S    ./parent_dies_first
```

รูปที่ 4 การเปลี่ยน PPID ของ Child หลัง Parent สิ้นสุด แสดงการ Re-parenting

Discussion เปรียบเทียบ 2.1 และ 2.2

กรณี Zombie คือ Child จบแล้วแต่ Parent เติมยังอยู่และยังไม่ reap Child ส่วนกรณี Parent จบก่อน Child นั้น Child ยังทำงานต่อและถูกระบบ Re-parent ไปยัง Process อื่น ดังนั้นสองสถานการณ์ต่างกันว่า Zombie ไม่ได้ execute ต่อแล้ว แต่ Orphan/Re-parented Child ยังสามารถทำงานต่อได้ ทั้งนี้ PPID ใหม่ไม่จำเป็นต้องเป็น 1 เสมอ ในเครื่องที่ใช้ทดลองพบค่า 4005 จึงรายงานตามค่าจริงจากระบบ

Problem 3: Maximum Number of Successful fork() Calls

- Find out how many processes you can **fork**. Write a program that forks until it cannot **fork** anymore and report how many times **fork()** is successfully called. Then run some programs that stay running in the system, measure the **fork** count again, stop the programs that were created, and measure the **fork** count again. Discuss the result.

Hint: Parent may **wait** for Child and only the Child continues to **fork**, so that the Process structure grows as a linear chain rather than an uncontrolled exponential **fork** tree.

Caution: This experiment can crash the system if **fork()** is not controlled carefully.

3.1 Source code และแนวทางควบคุมการทดลอง

Source code

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(void){
    long count = 0;

    while (1){
        pid_t pid = fork();

        if (pid < 0){
            perror("fork");
            printf("Successful fork() calls: %ld\n", count);
            fflush(stdout);
            break;
        }
        else if (pid == 0){
            count++;
            continue;
        }
        else{
            wait(NULL);
            break;
        }
    }

    return 0;
}
```

หลักการทำงานของโปรแกรม

โปรแกรมออกแบบให้เฉพาะ Child เป็นผู้ `fork()` ต่อ ส่วน Parent ของแต่ละระดับเรียก `wait(NULL)` แล้วหยุดสร้าง Child เพิ่ม ทำให้โครงสร้าง Process เป็นสาย P0 -> P1 -> P2 -> ... แทนการแตกกิ่งแบบ exponential `fork` bomb เมื่อ `fork()` คืนค่าติดลบ โปรแกรมจะใช้ `perror()` แสดงสาเหตุและแสดงจำนวน successful `fork()` ที่สะสมไว้

Methodology (system-limit check): เพื่อให้การทดลองทำซ้ำได้และลดความเสี่ยงต่อความไม่เสถียรของ VM ได้กำหนด temporary RLIMIT_NPROC = 900 ภายใน subshell เฉพาะช่วงการทดลอง โดย limit ปกติของ user ก่อนทดลองคือ 6169 การใช้ค่า 900 จึงเป็น controlled condition สำหรับเปรียบเทียบ Test A, B และ C และไม่ตีความค่า fork count ที่ได้ว่าเป็น absolute maximum ของ hardware ทั้งเครื่อง

3.2 Test A: Baseline

รันโปรแกรมภายใต้ temporary limit 900 โดยยังไม่สร้าง background process เพิ่ม ผลที่ได้คือ `fork()` สำเร็จ 330 ครั้งก่อนพบ “Resource temporarily unavailable”

```
wisitsuwannao@Ubuntu26LTS:~/CPE333/PS2/q3$ ps -elf -u "$USER" --no-headers | wc -l
820
wisitsuwannao@Ubuntu26LTS:~/CPE333/PS2/q3$ ulimit -u
6169
wisitsuwannao@Ubuntu26LTS:~/CPE333/PS2/q3$ (
    ulimit -u 900
    echo "Temporary limit:"
    ulimit -u
    ./fork_limit
)
Temporary limit:
900
fork: Resource temporarily unavailable
Successful fork() calls: 331
wisitsuwannao@Ubuntu26LTS:~/CPE333/PS2/q3$ ulimit -u
ps -elf -u "$USER" --no-headers | wc -l
6169
815
wisitsuwannao@Ubuntu26LTS:~/CPE333/PS2/q3$ (
    ulimit -u 900
    ./fork_limit
) 2>&1 | tee ../evidence/q3_test_a.txt
fork: Resource temporarily unavailable
Successful fork() calls: 330
```

รูปที่ 5 ผล Test A ภายใต้ controlled process limit 900 ได้ 330 successful fork() calls

3.3 Test B: เพิ่ม Background Process

สร้างคำสั่ง `sleep 300` จำนวน 20 Process ให้ค้างอยู่ในระบบและตรวจด้วย `jobs -p | wc -l` ได้จำนวน 20 จากนั้นรัน `fork_limit` ภายใต้ temporary limit 900 เหมือน Test A ผลที่ได้ลดลงเหลือ 299 successful `fork()` calls

```
wisitsuwannao@Ubuntu26LTS:~/CPE333/PS2/q3$ for i in {1..20}; do
    sleep 300 &
done
[1] 24922
[2] 24923
[3] 24924
[4] 24925
[5] 24926
[6] 24927
[7] 24928
[8] 24929
[9] 24930
[10] 24931
[11] 24932
[12] 24933
[13] 24934
[14] 24935
[15] 24936
[16] 24937
[17] 24938
[18] 24939
[19] 24940
[20] 24941
wisitsuwannao@Ubuntu26LTS:~/CPE333/PS2/q3$ jobs -p | wc -l
20
wisitsuwannao@Ubuntu26LTS:~/CPE333/PS2/q3$ (
    ulimit -u 900
    ./fork_limit
) 2>&1 | tee ../evidence/q3_test_b.txt
fork: Resource temporarily unavailable
successful fork() calls: 299
```

รูปที่ 6 ผล Test B เมื่อเพิ่ม `sleep 300` จำนวน 20 Process ได้ 299 successful `fork()` calls

3.4 Test C: หลังยุติ Background Process

หลังจาก Test B ใช้ `kill $(jobs -p)` และ `wait` เพื่อ terminate และเก็บ background jobs ที่สร้างขึ้น จากนั้นรัน `fork_limit` อีกครั้งภายใต้ temporary limit 900 ผลที่ได้คือ 326 successful `fork()` calls

```
wisitsuwannao@Ubuntu26LTS:~/CPE333/PS2/q3$ kill $(jobs -p)
wait
[2]   Terminated                  sleep 300
[3]   Terminated                  sleep 300
wisitsuwannao@Ubuntu26LTS:~/CPE333/PS2/q3$ jobs
```

รูปที่ 7 หลักฐานการ cleanup background Process ด้วย `kill $(jobs -p)` และ `wait`

```
wisitsuwannao@Ubuntu26LTS:~/CPE333/PS2/q3$ (
  ulimit -u 900
  ./fork_limit
) 2>&1 | tee ../evidence/q3_test_c.txt
fork: Resource temporarily unavailable
Successful fork() calls: 326
```

รูปที่ 8 ผล Test C หลัง cleanup background Process ได้ 326 successful `fork()` calls

สรุปผลและ Discussion

Test A (Baseline): 330 successful `fork()` calls

Test B (+20 `sleep` processes): 299 successful `fork()` calls

Test C (After cleanup): 326 successful `fork()` calls

ผลเป็นไปตามแนวโน้ม $N2 < N1$ และ $N3 \approx N1$ กล่าวคือ เมื่อมี Processอื่นใช้ทรัพยากรอยู่ก่อน ความสามารถในการสร้าง Process ใหม่จะลดลงและ `fork()` fail เร็วขึ้น หลัง terminate Process ที่เพิ่มเข้าไป Resource ถูกคืนกลับทำให้ค่าใน Test C กลับมาใกล้ Baseline อีกครั้ง จำนวนที่ลดลงไม่จำเป็นต้องเท่ากับจำนวน `sleep` ที่เพิ่มแบบหนึ่งต่อหนึ่ง เนื่องจาก background tasks และ resource accounting ของระบบสามารถเปลี่ยนระหว่างการทดลองได้

Problem 4: Pipe Communication between Child and Parent

4. Write a program that creates a pipe using `pipe()` and then spawns a new child process using `fork()`. The parent and child should communicate through the pipe. The sender must close the read-end and the receiver must close the write-end. Let the Child be the sender and the Parent be the receiver. The Child prints its PID and sends "Hello from child" with its PID. The Parent prints its PID and then prints the received message.

4.1 Source code

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(void){
    int fd[2];

    if (pipe(fd) == -1){
        perror("pipe");
        exit(1);
    }

    pid_t pid = fork();

    if (pid < 0){
        perror("fork");
        exit(1);
    }
    else if (pid == 0){
        close(fd[0]);

        pid_t child_pid = getpid();
        printf("Child: Child PID: %d\n", (int)child_pid);

        char message[100];
        snprintf(message, sizeof(message), "Hello from child PID: %d",
(int)child_pid);

        if (write(fd[1], message, strlen(message) + 1) == -1){
            perror("write");
            exit(1);
        }

        close(fd[1]);
        return 0;
    }
```

```

    }
    else{
        close(fd[1]);
        printf("Parent: Parent PID: %d\n", (int)getpid());

        char buffer[100];
        ssize_t bytes_read = read(fd[0], buffer, sizeof(buffer));

        if (bytes_read == -1){
            perror("read");
            exit(1);
        }

        printf("Parent: %s\n", buffer);
        close(fd[0]);
        wait(NULL);
    }

    return 0;
}

```

หลักการทำงานของโปรแกรม (Program Operation Description)

โปรแกรมเรียก `pipe(fd)` ก่อน `fork()` เพื่อให้ Parent และ Child inherit File Descriptor ของ Pipe เดียวกัน โดย `fd[0]` เป็น read-end และ `fd[1]` เป็น write-end หลัง `fork()` Child ทำหน้าที่ Sender จึง `close(fd[0])` และเขียนข้อความที่มี Child PID ไปยัง `fd[1]` ส่วน Parent เป็น Receiver จึง `close(fd[1])` แล้ว `read(fd[0])` เพื่อรับข้อความ

การปิดปลาย Pipe ที่ไม่ได้ใช้ช่วยให้ semantics ของ Pipe ถูกต้องและลดความสับสนเรื่อง EOF/การ block ในการทำงาน ภายหลัง Parent เรียก `wait(NULL)` เพื่อเก็บ Child หลังการสื่อสารเสร็จ

ผลการทดลอง (Output Results)

ทดลองรัน 3 ครั้งพบว่าทุกครั้ง Parent รับข้อความจาก Child ได้ถูกต้อง และ PID ที่อยู่ในข้อความ “Hello from child PID: 25839”, “Hello from child PID: 25841” และ “Hello from child PID: 25843” ตรงกับ Child PID ของรอบนั้น แม้บรรทัด Child/Parent ตอนเริ่มอาจมีลำดับตาม Scheduling ของระบบ

```
wisitsuwannao@Ubuntu26LTS:~/CPE333/PS2/q4$ for i in {1..3}; do
    echo "==== RUN $i ====="
    ./pipe_basic
done | tee ../evidence/q4_pipe_basic.txt
==== RUN 1 =====
Child: Child PID: 25839
Parent: Parent PID: 25838
Parent: Hello from child PID: 25839
==== RUN 2 =====
Child: Child PID: 25841
Parent: Parent PID: 25840
Parent: Hello from child PID: 25841
==== RUN 3 =====
Child: Child PID: 25843
Parent: Parent PID: 25842
Parent: Hello from child PID: 25843
```

รูปที่ 9 ผลการสื่อสารผ่าน Pipe ระหว่าง Child และ Parent จำนวน 3 ครั้ง

Discussion

ผลยืนยันว่า `pipe()` ที่สร้างก่อน `fork()` สามารถใช้เป็นช่องทาง IPC ระหว่าง Parent และ Child ได้ เพราะทั้งสอง Process inherit File Descriptor ที่อ้างถึง Pipe เดียวกัน Parent อาจเรียก `read()` ก่อน Child `write()` ได้ แต่ `read()` จะรอข้อมูลจน Child ส่งเข้ามา ซึ่งเป็นพฤติกรรมที่จะทดสอบโดยตรงใน Problem 5.2

Problem 5: Pipe Behavior Experiments

5. Test the situations where:

- 5.1. Sender tries to **read** back its own message.
- 5.2. Receiver reads before sender sends.
- 5.3. Sender sends several messages before receiver reads.

Write down what you observed from the experiment and discuss the results.

5.1 Sender tries to read back its own message

Source code

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(void){
    int fd[2];

    if (pipe(fd) == -1){
        perror("pipe");
        exit(1);
    }

    pid_t pid = fork();

    if (pid < 0){
        perror("fork");
        exit(1);
    }
    else if (pid == 0){
        char message[] = "Hello from child";
        char buffer[100];

        printf("Child: writing message to pipe\n");

        ssize_t bytes_written = write(fd[1], message, strlen(message) + 1);
        if (bytes_written == -1) { perror("write"); exit(1); }

        printf("Child: wrote %zd bytes\n", bytes_written);
        printf("Child: trying to read its own message\n");
```

```

        ssize_t bytes_read = read(fd[0], buffer, sizeof(buffer));
        if (bytes_read == -1){
            perror("read");
        }
        else{
            printf("Child: read %zd bytes\n", bytes_read);
            printf("Child received: %s\n", buffer);
        }

        close(fd[0]);
        close(fd[1]);
        return 0;
    }
    else{
        close(fd[0]);
        close(fd[1]);
        wait(NULL);
    }

    return 0;
}

```

ผลการทดลอง

Child เขียนข้อความ “Hello from child” ลง Pipe ได้ 17 bytes จากนั้น Child ซึ่งยังถือ read-end อยู่เรียก `read()` และอ่านกลับได้ 17 bytes พร้อมข้อความเดิมครบถ้วน

```

wisitsuwanna@Ubuntu26LTS:~/CPE333/PS2/q5$ ./pipe_read_own
Child: writing message to pipe
Child: wrote 17 bytes
Child: trying to read its own message
Child: read 17 bytes
Child received: Hello from child

```

รูปที่ 10 ผลการทดลอง Sender อ่านข้อความของตนเองกลับจาก Pipe

Discussion

ผลนี้แสดงว่า Pipe ไม่ได้กำหนดว่าข้อมูลที่เขียนโดย Process หนึ่งจะต้องถูกอ่านโดย Process อื่นเท่านั้น ใน experiment นี้ Child ยังเปิด read-end และ Parent จึงไม่อ่านข้อมูล จึงทำให้ Child อ่าน byte ที่ตัวเองเขียนไว้กลับมาได้ อย่างไรก็ตามไม่ควรสรุปว่า Sender จะอ่านข้อมูลตัวเองได้เสมอ เพราะถ้ามี Reader อื่นแข่งขันอ่าน ข้อมูลอาจถูก Reader อื่น consume ไปก่อน

5.2 Receiver reads before Sender sends

Source code

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(void){
    int fd[2];

    if (pipe(fd) == -1){
        perror("pipe");
        exit(1);
    }

    pid_t pid = fork();

    if (pid < 0){
        perror("fork");
        exit(1);
    }
    else if (pid == 0){
        close(fd[0]);

        printf("Child: waiting 3 seconds before sending...\n");
        fflush(stdout);
        sleep(3);

        char message[] = "Hello after 3 seconds";
        printf("Child: sending message now\n");
        fflush(stdout);

        ssize_t bytes_written = write(fd[1], message, strlen(message) + 1);
        if (bytes_written == -1) { perror("write"); exit(1); }

        printf("Child: wrote %zd bytes\n", bytes_written);
        close(fd[1]);
        return 0;
    }
    else{
        close(fd[1]);
        char buffer[100];

        printf("Parent: calling read() before child sends...\n");
        fflush(stdout);

        ssize_t bytes_read = read(fd[0], buffer, sizeof(buffer));
```

```

    printf("Parent: read() returned\n");

    if (bytes_read == -1){
        perror("read");
    }
    else{
        printf("Parent: read %zd bytes\n", bytes_read);
        printf("Parent received: %s\n", buffer);
    }

    close(fd[0]);
    wait(NULL);
}

return 0;
}

```

ผลการทดลอง

Parent เรียก `read()` ก่อน ขณะที่ Child จงใจ `sleep 3` วินาทีก่อนส่งข้อมูล ในช่วงนั้น Parent ยังไม่สามารถดำเนินการคำสั่งหลัง `read()` ต่อได้ เมื่อ Child `write` ข้อความ “Hello after 3 seconds” จำนวน 22 bytes แล้ว Parent จึง `read()` สำเร็จและได้รับ 22 bytes

```

wisitsuwannao@Ubuntu26LTS:~/CPE333/PS2/q5$ ./pipe_receive_first
Parent: calling read() before child sends...
Child: waiting 3 seconds before sending...
Child: sending message now
Child: wrote 22 bytes
Parent: read() returned
Parent: read 22 bytes
Parent received: Hello after 3 seconds

```

รูปที่ 11 ผลการทดลอง Receiver เรียก `read()` ก่อน Sender ส่งข้อมูล

Discussion

ผลดังกล่าวแสดง Blocking Read ของ Pipe แบบปกติ เมื่อ Pipe ยังไม่มีข้อมูลและยังมี `write-end` ที่เปิดอยู่ `read()` จะรอจนมีข้อมูลพร้อมอ่าน หลัง Child `write()` แล้ว event ที่ Parent รอจึงเกิดขึ้นและ Parent สามารถกลับมาทำงานต่อได้ พฤติกรรมนี้สอดคล้องกับแนวคิด Process State ที่ Process สามารถเข้าสู่ Blocked/Waiting state ระหว่างรอ I/O หรือ event

5.3 Sender sends several messages before Receiver reads

Source code

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(void){
    int fd[2];

    if (pipe(fd) == -1){
        perror("pipe");
        exit(1);
    }

    pid_t pid = fork();

    if (pid < 0){
        perror("fork");
        exit(1);
    }
    else if (pid == 0){
        close(fd[0]);

        const char *msg1 = "Message 1\n";
        const char *msg2 = "Message 2\n";
        const char *msg3 = "Message 3\n";

        ssize_t n1 = write(fd[1], msg1, strlen(msg1));
        ssize_t n2 = write(fd[1], msg2, strlen(msg2));
        ssize_t n3 = write(fd[1], msg3, strlen(msg3));

        if (n1 == -1 || n2 == -1 || n3 == -1){
            perror("write");
            exit(1);
        }

        printf("Child: first write = %zd bytes\n", n1);
        printf("Child: second write = %zd bytes\n", n2);
        printf("Child: third write = %zd bytes\n", n3);
        printf("Child: finished all writes\n");

        close(fd[1]);
        return 0;
    }
    else{
        close(fd[1]);
```

```

wait(NULL);

printf("Parent: child finished writing\n");
printf("Parent: reading from pipe now\n");

char buffer[256];
ssize_t bytes_read = read(fd[0], buffer, sizeof(buffer) - 1);

if (bytes_read == -1){
    perror("read");
    exit(1);
}

buffer[bytes_read] = '\0';
printf("Parent: read %zd bytes\n", bytes_read);
printf("Parent received:\n%s", buffer);

close(fd[0]);
}

return 0;
}

```

ผลการทดลอง

Child เรียก `write()` สามครั้ง โดยแต่ละครั้งเขียน 10 bytes รวม 30 bytes แล้วสิ้นสุดการเขียนก่อนที่ Parent จะ `read()` จากนั้น Parent เรียก `read()` เพียงครั้งเดียวและได้ 30 bytes ซึ่งประกอบด้วย Message 1, Message 2 และ Message 3 ต่อเนื่องกัน

```

wisitsuwannao@Ubuntu26LTS:~/CPE333/PS2/q5$ ./pipe_multiwrite
Child: first write = 10 bytes
Child: second write = 10 bytes
Child: third write = 10 bytes
Child: finished all writes
Parent: child finished writing
Parent: reading from pipe now
Parent: read 30 bytes
Parent received:
Message 1
Message 2
Message 3
wisitsuwannao@Ubuntu26LTS:~/CPE333/PS2/q5$ ./pipe_multiwrite | tee ../evidence/q5_3_multiwrite.txt
Child: first write = 10 bytes
Child: second write = 10 bytes
Child: third write = 10 bytes
Child: finished all writes
Parent: child finished writing
Parent: reading from pipe now
Parent: read 30 bytes
Parent received:
Message 1
Message 2
Message 3

```

รูปที่ 12 ผลการทดลอง Sender เขียนหลายข้อความก่อน Receiver อ่าน

Discussion

ผลนี้แสดงลักษณะของ Pipe ที่ทำงานเป็น byte stream: จำนวนครั้งของ `write()` ไม่จำเป็นต้องจับคู่หนึ่งต่อหนึ่งกับจำนวนครั้งของ `read()` เมื่อข้อมูลหลายชุดอยู่ใน Pipe แล้วและ buffer ที่ `read()` มีขนาดเพียงพอ การ `read()` หนึ่งครั้งสามารถดึง byte ที่มาจากหลาย `write()` มารวมกันได้ ดังนั้นโปรแกรมที่ต้องการแบ่ง Message Boundary ควรกำหนด protocol หรือรูปแบบข้อมูลของตนเองแทนการสมมติว่า `write()` หนึ่งครั้งจะเท่ากับ `read()` หนึ่งครั้งเสมอ